

Sorting

Sorting is the process of *arranging or ordering* (ascendingly or descendingly) a collection of *items* such that each item and its successor satisfy a prescribed relationship. The items can be *simple values*, such as *integers* and *reals*, or more *complex types*, such as *student records* or *dictionary entries*. In either case, the ordering of the items is based on the value of a *sort key*. The *key* is the *value itself* when sorting simple types or it can be a specific component or a combination of components when sorting complex types.

Example: Sorting [5, 2, 9, 1] in ascending order → [1, 2, 5, 9]

Sorting [5, 2, 9, 1] in descending order → [9, 5, 2, 1]

Bubble Sort

A simple solution to the sorting problem is the *bubble sort* algorithm, which *rearranges* the values by *iterating over the list multiple times*, causing larger values to bubble to the top or end of the list. **Bubble Sort** is the simplest sorting algorithm that *repeatedly compares adjacent elements* in a *list* and *swaps them* if they are in the *wrong order*.

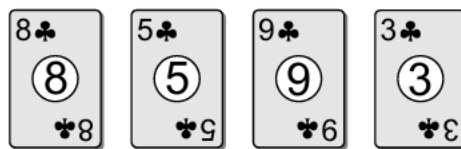
- ✚ The largest element "bubbles up" to the *end* in each pass.
- ✚ After the *first pass*, the *biggest element* is at the correct position; after the second pass, the second-biggest is in place, and so on.

Example (Sorting in *ascending* order):

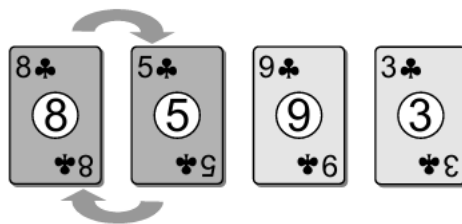
Suppose a **List** have values: [5, 3, 8, 4]

- + Compare 5 & 3 → swap → [3, 5, 8, 4]
- + Compare 5 & 8 → no swap → [3, 5, 8, 4]
- + Compare 8 & 4 → swap → [3, 5, 4, 8]
- + End of first pass, 8 is in correct place

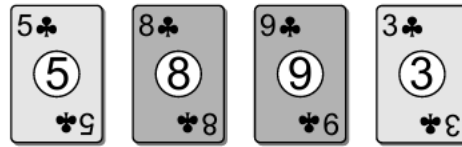
Suppose we have four playing cards (*all of the same suit*) that we want to order from smallest to largest face value. We start by laying the cards out face up on a table as shown here.



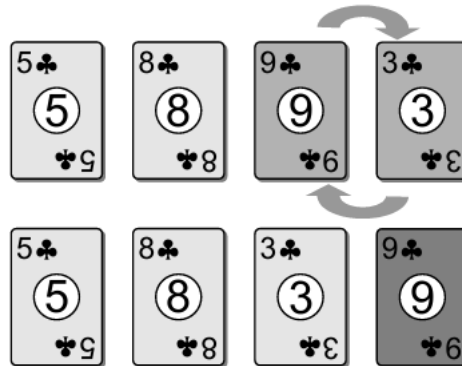
The algorithm requires multiple passes over the cards, with each pass starting at the first card and ending one card earlier than on the previous iteration. During each pass, the cards in the first and second positions are compared. If the first is larger than the second, the two cards are swapped.



Next, the cards in positions two and three are compared. If the first one is larger than the second, they are swapped. Otherwise, we leave them as they were.

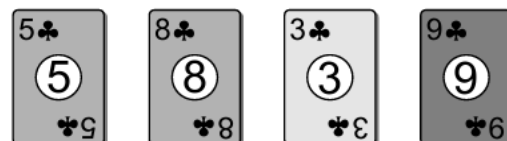


This process continues for each successive pair of cards until the card with the largest face value is positioned at the end.

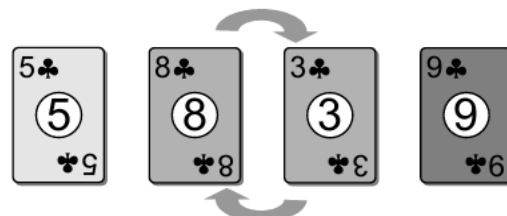


The next two passes over the cards are illustrated below. In the second pass the card with the second largest face value is positioned in the next-to-last position. In the third and final pass, the first two cards will be positioned correctly.

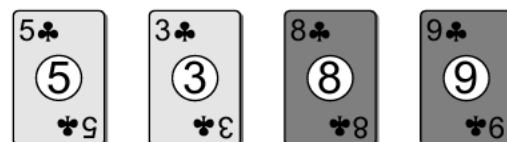
(Pass 2) Repeat the process on the first three cards. Compare the 5 and 8. Since 5 is less than 8, leave them as is.



Compare the 8 and 3. Since 8 is larger than 3, swap the two cards.

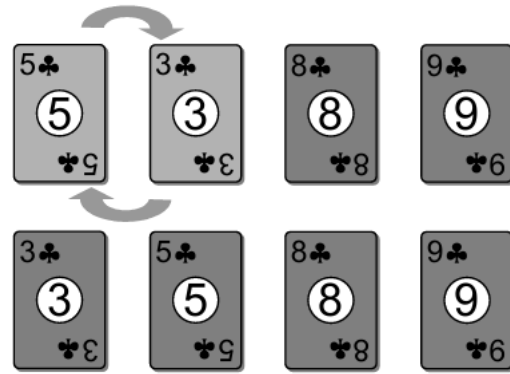


The second largest card (8) is now in its ordered position.



(Pass 3) Repeat the process on the first two cards. Compare the 5 and 3. Since 5 is larger than 3, swap the two cards.

After swapping the two cards, all of the cards are now in their proper order, from smallest to largest.



The efficiency of the bubble sort algorithm only depends on the number of keys in the array and is independent of the specific values and the initial arrangement of those values. To determine the efficiency, we must determine the total number of iterations performed by the inner loop for a sequence containing n values. The outer loop is executed $n-1$ times since the algorithm makes $n-1$ passes over the sequence. The number of iterations for the inner loop is not fixed, but depends on the current iteration of the outer loop. On the first pass over the sequence, the inner loop executes $n-1$ times; on the second pass, $n-2$ times; on the third, $n-3$ times, and so on until it executes once on the last pass. The total number of iterations for the inner loop will be the sum of the first $n-1$ integers, which equals resulting in a run time of $O(n^2)$. Bubble sort is considered one of the most in-efficient sorting algorithms due to the total number of swaps required. Given an array of keys in reverse order, a swap is performed for every iteration of the inner loop, which can be costly in practice.

1- Where do we apply Bubble Sort?

Bubble Sort is mostly used in:

- + **Teaching & learning:** To understand sorting and algorithm concepts.
- + **Small datasets:** When the dataset is tiny (10–20 elements).
- + **Almost sorted arrays:** Works better if the list is nearly sorted.
- + **Simple embedded systems:** Where memory is very limited, and simplicity is preferred over speed.

2- Advantages of Bubble Sort

- + Simple to *understand* and *implement*.
- + No *extra memory* required (works in-place).
- + Works well on *small* or *nearly sorted* datasets.
- + Can be easily *optimized* (stop early if no swaps happen in a pass):

3- Disadvantages of Bubble Sort

- + Very slow for large datasets → Time Complexity = $O(n^2)$.
- + Performs many unnecessary comparisons & swaps.
- + Not efficient compared to advanced algorithms like Quick Sort, Merge Sort, Heap Sort, etc.

4- Implementation of Bubble Sort

1) First Step is to Make *Function of Bubble Sort*.

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        swapped = False  
        for j in range(n - i - 1):  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
                swapped = True  
                print(f"Swap {arr[j]} and {arr[j + 1]}: {arr}")  
        print(f"Pass {i + 1}: {arr}\n")  
        if not swapped: # Early exit if no swaps  
            break  
    return arr
```

2) Call *Bubble Sort Function* for Data

```
# Example usage:  
data = [64, 34, 25, 12, 22, 11, 90]  
print(f"Unsorted list: {data}\n")  
bubble_sort(data)  
print(f"Sorted list: {data}")
```

3) Call **Bubble Sort** *Function* to **Sort Stack or Queue**

```
def bubble_sort(self):
    n = len(self.stack)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if self.stack[j] > self.stack[j + 1]:
                self.stack[j], self.stack[j + 1] = self.stack[j + 1], self.stack[j]
                swapped = True
                print(self.stack) # only print when a swap happens
        print("")
        print(f"After pass {i+1}: {self.stack}")
        if not swapped: # optimization: stop if already sorted
            break
```

Hint: For the *sorting* of the *queue*, just *call* the existing *queue* instead of the *stack*.

5- Practice Questions

Below are the **5 practice questions** based on the **Bubble Sort** implementation that will help you to learn and practice **Bubble Sort** effectively:

1. Sorting Patient Wait Times in a Clinic

You are developing a system for a medical clinic where patients arrive with estimated wait times (in minutes). The clinic wants to prioritize by sorting the wait times in ascending order using Bubble Sort to handle urgent cases first.

- ✚ Given a list like **[45, 10, 30, 5, 60]**, implement **Bubble Sort** to sort it.
- ✚ Print the list **after each pass** (no need for swap prints), and include the early termination optimization with a **swapped flag**.
- ✚ Test with the example list and output the final sorted list.

2. Organizing Inventory Prices in a Store

A **small retail store** tracks product **prices** in a list, such as [19.99, 5.50, 12.00, 8.75, 25.00], but they need to sort them in **ascending order** for a discount report using **Bubble Sort**.

- ✚ Write the Bubble Sort function without the swapped optimization (to always complete all passes).
- ✚ Print the list after each swap and each pass.
- ✚ Use the example list, and ensure the function modifies the list in place

3. Ranking Exam Scores for a Class

In a classroom, a teacher has a list of **student exam scores**: [85, 92, 78, 95, 88, 70].

- ✚ To rank students, sort the scores in **descending order** (modify the comparison in Bubble Sort to if `arr[j] < arr[j + 1]` for swaps).
- ✚ Include swap prints showing which elements are swapped and the list state, plus pass prints.
- ✚ Use the **swapped flag** for optimization.
- ✚ Test with the example and print the final sorted list.

4. Sorting Log Entries from a Stack (Incorporating Stack)

You are analyzing **server logs** stored in a **stack** (LIFO order, using **collections.deque** as a stack). The logs are error codes like [404, 500, 200, 503, 200] pushed onto the stack in the order they occurred.

- ✚ To process them in sorted **ascending order**, **pop** all elements from the stack into a **list**, sort the **list** using **Bubble Sort** (with swap and pass prints, no swapped optimization), and then **push** the sorted elements back onto a **new stack**.
- ✚ Write the code to do this, test with the example, and **print** the **final stack** contents (by popping them out to display).

5. Prioritizing Tasks in a Queue (Incorporating Queue)

A **task management app** uses a **queue** (FIFO, using **collections.deque** as a queue) to hold task priorities: [3, 1, 4, 2, 5] (lower number = higher priority, enqueued in arrival order).

- ✚ To reprioritize, **dequeue** all tasks into a **list**, sort the **list** in **ascending** order using **Bubble Sort** (with swapped optimization, pass prints only), and **enqueue** the sorted priorities back into the **queue**.
- ✚ Implement this, test with the example, and **print** the **final queue** by **dequeuing** all elements to display.